



MANUAL DE ESTUDIO DEL EQUIPO

CUARENTENA

Todo lo que necesitamos saber para hacer nuestro videojuego de supervivencia

Juego de supervivencia con zombies · pixel art 2D · Godot 4
Plan de estudio de 1 semana · Núcleo común + rutas por rol
Equipo de 3 · Proyecto sin fines de lucro

ANTES DE EMPEZAR

Cómo usar este libro

Este manual está hecho a la medida de **nuestro** juego. No es un curso genérico de internet: cada tema se explica y después se muestra **cómo aparece en el código que ya tenemos** (en la carpeta `game/` del repo). La idea es que en una semana lleguemos con el conocimiento fresco para empezar a trabajar sin estar perdidos.

Los 3 roles

Somos 3 y nos repartimos el trabajo. Cada uno tiene una **ruta** propia, pero **todos** arrancamos por el mismo núcleo común:

- **PROGRAMACIÓN** — hace que el juego funcione: movimiento, zombies, crafeo, guardado. (GDScript en Godot.)
- **ARTE** — dibuja el mundo: tiles del mapa, personaje, zombies, objetos, animaciones y la interfaz.
- **DISEÑO + AUDIO** — piensa que el juego sea divertido y justo (balance, niveles) y arma el sonido y la música.

No es rígido: si a alguien le interesa otra ruta, la lee igual. Y los tres deberían entender lo básico de todo para poder hablar el mismo idioma.

Cómo estudiar cada capítulo

Cada capítulo tiene tres partes: **1)** la explicación del concepto, **2)** un recuadro "*En nuestro juego*" que lo conecta con un archivo real del proyecto, y **3)** un *mini-ejercicio* para fijar lo aprendido. Hagan los ejercicios: se aprende programando/dibujando, no solo leyendo.

Recordá el objetivo

No hace falta memorizar todo. Con entender *de qué se trata cada cosa* y saber *dónde buscarla* ya alcanza para arrancar. El conocimiento profundo se gana trabajando en el juego, no antes.

Índice

Parte 0 · Núcleo común **TODOS**

1. Qué es un videojuego por dentro
2. Nuestro juego en una página
3. Git y GitHub para trabajar en equipo
4. Godot 4: el editor y cómo está armado nuestro proyecto
5. Notion y cómo nos organizamos

Parte 1 · Ruta Programación **PROGRAMACIÓN**

6. GDScript desde cero
7. Nodos, escenas y señales
8. Input, movimiento y cámara
9. Colisiones, capas y máscaras
10. Componentes y arquitectura

- 11. Inteligencia artificial y máquinas de estado
- 12. Datos, JSON y guardado de partida
- 13. TileMaps: construir el mundo

Parte 2 · Ruta Arte **ARTE**

- 14. Fundamentos de pixel art
- 15. Aseprite en la práctica
- 16. Tilesets y autotiling
- 17. Personajes y animación
- 18. Interfaz (UI/HUD) con Figma

Parte 3 · Ruta Diseño + Audio **DISEÑO + AUDIO**

- 19. Game design: que sea divertido y justo
- 20. Diseño de niveles y del mapa
- 21. Audio: sonido y música (y el ruido como mecánica)

Parte 4 · Avanzado **TODOS (un vistazo)**

- 22. Buenas prácticas y organización del código
- 23. Multijugador: lo más difícil
- 24. Seguridad aplicada
- 25. Exportar y publicar en itch.io

Apéndices

- A. Cronograma de estudio de 1 semana
- B. Glosario de términos
- C. Recursos gratis para seguir aprendiendo

Núcleo común

Estos 5 capítulos los estudian los tres, sin importar el rol. Son la base para entender de qué hablamos cuando hablamos de "hacer un juego" y para poder trabajar juntos sin pisarnos.

Qué es un videojuego por dentro

Un videojuego parece magia, pero por dentro es sorprendentemente simple: es un programa que **repite lo mismo muchísimas veces por segundo**. A esa repetición se le llama **game loop** (bucle de juego).

El game loop y los frames

Cada vuelta del bucle es un **frame** (cuadro). En cada frame el juego hace tres cosas, en orden:

1. **Leer la entrada:** ¿qué teclas apretó el jugador?
2. **Actualizar el estado:** mover al personaje, mover a los zombies, restar hambre, chequear colisiones...
3. **Dibujar:** pintar en la pantalla cómo quedó todo.

Esto pasa unas **60 veces por segundo** (60 FPS = *frames per second*). Como es tan rápido, nuestro ojo ve movimiento fluido, igual que en el cine. Un motor de juego (como Godot) se encarga de correr ese bucle por vos: vos solo escribís *qué pasa en cada frame*.

El vocabulario básico

Término	Qué es	Ejemplo en nuestro juego
Sprite	Una imagen 2D que se dibuja en pantalla	El personaje, un zombie, una fruta
Tile	Un "azulejo" cuadrado que se repite para armar el mapa	Pasto, camino, pared
Entidad	Un "objeto" del juego con comportamiento	El jugador, un zombie, un ítem
Colisión	Detectar cuándo dos cosas se tocan	El zombie te alcanza; agarrás comida
Escena	Un conjunto de piezas armadas que forman algo (un nivel, un personaje)	La escena del jugador, la del mundo
Física	Cálculo de movimiento y choques	Que no atraveses paredes

En nuestro juego

Todo lo que ves en las capturas —vos, el zombie, la comida, el mapa— son *entidades* hechas de *sprites*, moviéndose y chocando dentro de un *game loop* que corre 60 veces por segundo. Ahora mismo el arte son cuadrados de colores (placeholder), pero el motor por debajo ya hace todo esto.

Mini-ejercicio

Agarrá cualquier juego que tengas y tratá de identificar, mirándolo: ¿cuáles son los sprites? ¿cuáles son los tiles del fondo? ¿cuándo hay una colisión? Escribí 3 ejemplos de cada uno. Es entrenar el ojo para "ver" un juego como lo ve quien lo hace.

Nuestro juego en una página

Antes de aprender a construirlo, hay que tener clarísimo **qué** estamos construyendo. Este es el resumen. (El detalle completo está en `docs/GDD.md` y `docs/MASTER_PLAN.md` del repo.)

La idea

Cuarentena es un juego de **supervivencia con zombies**, en **pixel art 2D** con vista desde arriba (top-down), para PC. Inspirado en *Project Zomboid*, *DayZ* y *The Forest*. Se juega solo, y más adelante en cooperativo.

El loop de juego (lo que el jugador hace todo el tiempo)

- **Explorás** un mundo abierto: pueblo, bosque, costa.
- **Juntás recursos**: looteás casas, cazás, pescás, talás.
- **Sobrevivís**: manejas hambre, sed y salud.
- **Construís tu base** donde quieras en el mapa.
- **Evitás o enfrentás zombies**, que te detectan por lo que **ven** y por el **ruido** que hacés.

Los 3 pilares de diseño

1. **Libertad de asentamiento**: no hay una base obligatoria; el mapa abierto es el punto de partida.
2. **Tensión sostenida, no sustos baratos**: el peligro viene de administrar recursos y de manejar el ruido/visibilidad, no de sustos scriptados.
3. **Alcance controlado**: primero un mapa chico que funcione bien, antes de prometer un mundo gigante.

En nuestro juego (lo que ya está hecho)

El prototipo ya tiene andando: movimiento con sigilo (caminar/correr/agacharse, cada uno hace distinto ruido), un zombie con inteligencia que te detecta por visión y por ruido y te persigue, hambre y salud, juntar y comer comida, y una pantalla de mapa. Todo con arte placeholder. **Sobre esta base vamos a construir el resto.**

Mini-ejercicio

Con tus palabras, explicá el juego a alguien que no sabe nada, en 3 oraciones. Si podés hacerlo, entendiste el norte. Guardá esas 3 oraciones: son la base del "pitch" del juego.

Git y GitHub para trabajar en equipo

Somos 3 tocando los mismos archivos. Sin una herramienta que ordene eso, es un caos garantizado (se pisan cambios, se pierde trabajo). Esa herramienta es **Git**, y **GitHub** es la web donde guardamos el proyecto compartido. **Esto lo tienen que saber los tres, sí o sí.**

Las ideas clave

- **Repositorio (repo):** la carpeta del proyecto con todo su historial. El nuestro: `github.com/lorenzoengraf10-dot/z`.
- **Commit:** una "foto" guardada de tus cambios, con un mensajito que explica qué hiciste.
- **Push:** subir tus commits a GitHub para que los demás los vean.
- **Pull:** bajar los cambios que subieron los demás.
- **Rama (branch):** una línea de trabajo paralela. Nosotros por ahora trabajamos todos sobre `main`.

El flujo de trabajo diario (memorizá esto)

```
# 1. Antes de empezar a trabajar, bajá lo último:
git pull origin main

# 2. Trabajás, hacés cambios en Godot/Aseprite...

# 3. Guardás tus cambios en un commit:
git add .
git commit -m "Agrego animación de caminar del personaje"

# 4. Subís tus cambios:
git push origin main
```

⚠ La regla de oro para no pisarse

Siempre hacé `git pull` **antes** de empezar y `git push` **apenas terminás** algo. Si dos personas editan el mismo archivo a la vez, Git avisa de un "conflicto": no es grave, pero es más fácil evitarlo trabajando cada uno en cosas distintas y sincronizando seguido. **Coordinen en Notion quién toca qué.**

💡 Consejo

Instalen **GitHub Desktop** (app gratis con botones, sin comandos) si la terminal les intimida. Hace lo mismo: pull, commit, push, con clicks. Para arrancar es perfecto.

✏ Mini-ejercicio

Creen su cuenta de GitHub, pidan acceso al repo, y practiquen el ciclo completo: cloná el repo, editá el `README.md` agregando tu nombre, y hacé commit + push. Cuando los tres aparezcan en el historial, ya saben lo esencial de Git.

Godot 4: el editor y cómo está armado nuestro proyecto

Godot es el motor con el que hacemos el juego. Es gratis, libre, y perfecto para 2D. Lo bajás de `godotengine.org`, es un solo archivo que se ejecuta directo (no se instala).

Las 2 ideas más importantes de Godot: nodos y escenas

Godot arma **todo** con dos conceptos:

- **Nodo:** una pieza que hace *una* cosa. Hay nodos para mostrar una imagen, para detectar colisiones, para reproducir un sonido, para moverse con física, etc.
- **Escena:** varios nodos armados juntos como un árbol (uno principal y otros que "cuelgan" de él). Una escena puede ser un personaje, un enemigo, un menú o un nivel entero. Y las escenas se pueden meter dentro de otras (el jugador vive dentro de la escena del mundo).

Cómo está armado NUESTRO proyecto

Todo vive en la carpeta `game/`. Así se organiza:

Carpeta / archivo	Qué es
<code>project.godot</code>	El archivo que abrís en Godot para cargar todo el proyecto
<code>scenes/</code>	Las escenas: <code>Main</code> (el mundo), <code>Player</code> , <code>Zombie</code> , <code>Pickup</code> y la UI
<code>scripts/</code>	El código (GDScript): <code>player.gd</code> , <code>zombie.gd</code> , etc.
<code>data/</code>	Datos del juego, como <code>world_map.json</code> (las ubicaciones del mapa)
<code>assets/</code>	El arte y el sonido (por ahora vacío, con placeholders)

En nuestro juego

La escena `Main.tscn` es el mundo. Adentro tiene una instancia de `Player.tscn` (con una cámara que lo sigue), un `Zombie.tscn`, varios `Pickup.tscn` (la comida), el `HUD` (las barras) y el `MapScreen` (el mapa). Cada una de esas escenas tiene su propio script que le da comportamiento. Abrí `Main.tscn` en Godot y mirá el árbol de nodos del panel izquierdo: vas a ver exactamente esto.

Cómo probarlo

Abrí `game/project.godot` en Godot y apretá **F5**. Vas a poder jugar el prototipo: moverte, esquivar al zombie, juntar comida y abrir el mapa con M. Los controles están en `game/README.md`.

Mini-ejercicio

Abrí el proyecto en Godot, corré el juego con F5, y después abrí `Main.tscn`. Identificá en el árbol de nodos: ¿dónde está el jugador? ¿dónde el zombie? Hacé click en el nodo `Player` y mirá sus propiedades en el panel derecho.

Notion y cómo nos organizamos

Un proyecto de 3 personas necesita un solo lugar donde estén **todas las tareas**: qué hay que hacer, quién lo hace y en qué estado está. Usamos **Notion** (gratis).

Un tablero simple alcanza

Con un tablero tipo "kanban" de 4 columnas ya funciona muy bien:

Por hacer	En progreso	En revisión	Hecho
Tareas que nadie empezó	Lo que alguien está haciendo <i>ahora</i>	Terminado, falta que otro lo pruebe	Listo y probado

Cada tarjeta debería tener: un título claro ("Dibujar tileset de bosque"), un responsable, y a qué rol/parte pertenece.

Por qué importa tanto

Además de no olvidarse cosas, el tablero **evita que dos toquen lo mismo** (que en Git generaría conflictos). Si ves que una tarea está "en progreso" y tiene dueño, no la agarres: agarrá otra. Regla simple: cada uno mueve sus propias tarjetas y avisa cuando algo pasa a "En revisión".

Mini-ejercicio

Entre los tres, creen el Notion y carguen las primeras 10 tareas que se les ocurran del juego (da igual si están bien priorizadas). Repártanse 2-3 cada uno moviéndolas a "En progreso". Ya están organizados.

PARTE 1 · PROGRAMACIÓN

Ruta Programación

Esta ruta es para quien haga que el juego *funcione*. Vamos de lo más básico (escribir código) a lo más complejo (inteligencia artificial y guardado), siempre mostrando el código real que ya tenemos. Lenguaje: **GDScript**, el lenguaje propio de Godot, muy parecido a Python.

GDScript desde cero

GDScript es amigable: sin punto y coma, con indentación (sangría) para marcar bloques. Si sabés algo de Python, ya lo conocés medio.

Variables y tipos

Una **variable** es una cajita con un nombre que guarda un valor.

```
var vida = 100           # un número entero
var velocidad = 90.0     # un número con decimales
var nombre = "Superviviente" # texto (string)
var esta_vivo = true     # verdadero/falso (booleano)
```

Podés (y conviene) decir de qué tipo es cada una con `:` — ayuda a evitar errores:

```
var vida: int = 100
var velocidad: float = 90.0
```

Funciones

Una **función** es un bloque de código con nombre que hace una tarea. Se define con `func`:

```
func saludar(nombre):
    print("Hola " + nombre)

saludar("Lorenzo") # imprime: Hola Lorenzo
```

Decisiones (if) y repeticiones (for)

```
if vida <= 0:
    print("Moriste")
elif vida < 30:
    print("Cuidado, poca vida")
else:
    print("Todo bien")

for i in range(3):
    print("Zombie número " + str(i))
```

Listas y diccionarios

```
var inventario = ["madera", "piedra", "comida"] # lista (array)
var stats = { "vida": 100, "hambre": 80 }       # diccionario
print(stats["vida"]) # imprime 100
```

En nuestro juego

En `player.gd`, el inventario es justamente un diccionario: `var inventory := {}`. Cuando juntás comida, se hace `inventory["comida"] = inventory.get("comida", 0) + 1`. Eso es exactamente lo que acabás de aprender.

Mini-ejercicio

En Godot, creá un script nuevo y escribí una función `calcular_daño(vida, golpe)` que reste el golpe a la vida, imprima la vida restante, y si queda en 0 o menos imprima "Muerto". Probala con varios valores.

Nodos, escenas y señales

Ya vimos que Godot arma todo con nodos. Cada script se "engancha" a un nodo y le da comportamiento. Lo primero de cada script dice de qué tipo de nodo hereda:

```
extends CharacterBody2D    # este script controla un cuerpo con física 2D
```

Las funciones especiales que Godot llama solo

Función	Cuándo se ejecuta
<code>_ready()</code>	Una vez, al aparecer el nodo. Para inicializar cosas.
<code>_process(delta)</code>	Cada frame. Para lógica general.
<code>_physics_process(delta)</code>	Cada frame de física (ritmo fijo). Para mover cuerpos.
<code>_unhandled_input(event)</code>	Cuando hay una entrada (tecla, click).

`delta` es cuánto tiempo pasó desde el frame anterior (en segundos). Sirve para que el movimiento sea igual de rápido en cualquier compu, más allá de los FPS.

Señales: cómo se avisan las cosas entre sí

Una **señal** es un aviso que un nodo emite cuando pasa algo, y otros nodos pueden "escuchar". Es la forma de que las piezas se comuniquen sin estar pegadas entre sí.

En nuestro juego

El componente de necesidades (`needs_component.gd`) emite señales cuando cambian la salud o el hambre:

```
signal health_changed(current, maximum)
signal hunger_changed(current, maximum)
```

Y el HUD (`hud.gd`) las "escucha" para actualizar las barras:

```
needs.health_changed.connect(_on_health_changed)
```

Así el HUD se entera de los cambios *sin* que el jugador tenga que conocer al HUD. Están desacoplados: es una de las mejores prácticas del oficio.

Mini-ejercicio

Abrí `hud.gd` y seguí el recorrido: ¿de dónde saca el jugador? ¿a qué señales se conecta? ¿qué función corre cuando cambia el hambre? No hace falta que lo escribas: entenderlo leyéndolo ya es un montón.

Input, movimiento y cámara

Mover al personaje es "leer las teclas y cambiar su posición cada frame".

Acciones de input

En vez de preguntar por la tecla "W" directamente, se definen **acciones** ("move_up") y se les asignan teclas. Así, cambiar los controles no obliga a tocar el código del jugador.

En nuestro juego

El archivo `input_setup.gd` registra las acciones al arrancar (moverse con WASD y flechas, correr con Shift, agacharse con C, interactuar con E). Y `player.gd` las usa así:

```
func _physics_process(_delta):
    var input_dir = Input.get_vector("move_left", "move_right", "move_up", "move_down")
    var velocidad = walk_speed
    if Input.is_action_pressed("crouch"):
        velocidad = crouch_speed # agachado: más lento
    elif Input.is_action_pressed("run"):
        velocidad = run_speed # corriendo: más rápido
    velocity = input_dir * velocidad
    move_and_slide() # Godot mueve y resuelve choques
```

`get_vector` devuelve una dirección (ej. "arriba-derecha"), `velocity` es la velocidad del cuerpo, y `move_and_slide()` lo mueve deslizándose contra las paredes.

La cámara

Un nodo `Camera2D` puesto *dentro* del jugador hace que la cámara lo siga automáticamente. En nuestra `Main.tscn`, la cámara cuelga del `Player` con un zoom 2x (para que el pixel art se vea grande).

Mini-ejercicio

Corré el juego y probá los tres modos: caminar, correr (Shift) y agacharse (C). Después, en `player.gd`, cambiá `run_speed` a un número más grande, guardá, y volvé a correr el juego. Acabás de modificar el comportamiento del juego.

Colisiones, capas y máscaras

Las colisiones deciden qué choca con qué. En 2D, cada cuerpo tiene una forma de colisión (un rectángulo, un círculo) y pertenece a **capas**.

Capas y máscaras (la parte que confunde a todos al principio)

- **Capa (layer):** "en qué grupo estoy". Ej: el jugador está en la capa 2.
- **Máscara (mask):** "con qué grupos choco / a qué grupos detecto".

Ejemplo: si el jugador está en la capa 2 y las paredes en la capa 1, el jugador pone su *máscara* en 1 para chocar con paredes.

En nuestro juego

Usamos: **capa 1** = paredes, **capa 2** = jugador, **capa 4** = zombies. Así el "rayo de visión" del zombie (`VisionRay`) tiene su máscara solo en la capa 1: si el rayo choca con algo, es una pared que le tapa la vista. Y la comida (`Pickup` , un `Area2D`) tiene la máscara en la capa 2 para detectar solo al jugador cuando le pasás por encima.

Area2D vs CharacterBody2D

CharacterBody2D es un cuerpo sólido que se mueve y choca (el jugador, el zombie). **Area2D** es una zona que solo *detecta* cuando algo entra, sin frenarlo (la comida, un gatillo, una trampa). Elegir bien entre estos dos resuelve el 90% de los casos.

Mini-ejercicio

Abrí `Pickup.tscn` y mirá sus propiedades de colisión (`collision_layer` y `collision_mask`). Después leé `pickup.gd`: ¿qué pasa exactamente cuando el jugador la toca? Seguí la función `_on_body_entered`.

Componentes y arquitectura

A medida que el juego crece, meter TODO en un solo script se vuelve inmanejable. La solución: dividir en **componentes**, piezas reutilizables que hacen una cosa.

En nuestro juego

`needs_component.gd` maneja salud y hambre, y es un componente aparte, no está metido dentro del jugador. ¿Por qué? Porque mañana un NPC amigo también puede tener hambre y salud: le colgamos el *mismo* componente y listo. El jugador solo le "pide" cosas:

```
needs.damage(8.0)    # cuando un zombie te pega
needs.eat(30.0)      # cuando comés
```

El componente se encarga de bajar el hambre con el tiempo y, si llega a 0, empezar a restar salud. Todo eso encapsulado en un solo lugar.

La idea grande: bajar de nivel de a poco

Un buen código se lee como capas: el jugador no sabe *cómo* funciona la salud por dentro, solo pide `damage()`. El HUD no sabe cómo se calcula el hambre, solo escucha la señal. Cada pieza esconde su complejidad. Eso hace que 3 personas puedan trabajar sin romper lo del otro.

Mini-ejercicio

Leé `needs_component.gd` entero (es corto). Identificá: ¿dónde baja el hambre sola? ¿dónde empieza el daño por inanición? ¿qué señal se emite al morir? Es el ejemplo perfecto de un componente bien hecho.

Inteligencia artificial y máquinas de estado

La "IA" de un enemigo no es magia: es un conjunto de reglas y **estados**. Una **máquina de estados** dice: "el enemigo puede estar en uno de varios estados, y cambia según lo que pasa".

En nuestro juego

El zombie (`zombie.gd`) tiene tres estados:

```
enum State { WANDER, CHASE, ATTACK }  
# WANDER = deambula    CHASE = te persigue    ATTACK = te pega
```

Cada frame, el zombie ejecuta la lógica del estado en el que está, y decide si cambiar:

```
match state:  
  State.WANDER:  
    _do_wander(delta)  
    if player != null:      # si te detectó...  
      state = State.CHASE  # pasa a perseguirte  
  State.CHASE:  
    _do_chase(delta, player)  
  State.ATTACK:  
    _do_attack(delta, player)
```

Cómo te detecta: visión y oído

El zombie te encuentra de dos maneras (función `_detect_player`):

- **Oído:** si estás dentro de tu propio "radio de ruido" (que depende de si caminás, corrés o te agachás), te escucha aunque no te vea.
- **Visión:** si estás dentro de un cono al frente suyo y no hay una pared tapando (eso lo chequea el `VisionRay`).

Esto es exactamente lo que se ve en la captura del gameplay: el círculo punteado (ruido) y el triángulo (visión).

Por qué esto es tan poderoso

Con solo tres estados y dos formas de detección ya tenés un enemigo que se siente "inteligente". La mayoría de las IA de juegos son máquinas de estado como esta, solo que con más estados. Dominar este patrón te sirve para enemigos, NPCs, puertas, trampas... casi todo.

Mini-ejercicio

Abrí `zombie.gd` y encontrá `lose_interest_time`. Es cuántos segundos persigue el zombie después de perderte de vista. Cambialo a 10 segundos, corré el juego, y sentí la diferencia (¡ahora es mucho más insistente!). Después dibujá en papel el diagrama de los 3 estados con flechas de qué lo hace cambiar de uno a otro.

Datos, JSON y guardado de partida

No todo el juego es código: mucho es **datos**. Las ubicaciones del mapa, las recetas de crafeo, las estadísticas de cada objeto... conviene guardarlas en archivos aparte, no "hardcodeadas" en el código. El formato más común es **JSON**: texto ordenado en pares "nombre: valor".

En nuestro juego

El mapa lee sus lugares desde `data/world_map.json`. Cada lugar es así:

```
{
  "name": "Refugio Central",
  "type": "hub",
  "position": [0.5, 0.42],
  "description": "Asentamiento fortificado principal."
}
```

Y `map_screen.gd` lo lee y dibuja los puntos. Ventaja enorme: para **agregar o mover un lugar del mapa no hay que tocar código**, solo editar el JSON. El artista o el diseñador pueden hacerlo sin saber programar.

Guardar la partida

Guardar es lo mismo al revés: tomás el estado del juego (posición del jugador, vida, inventario), lo convertís a JSON y lo escribís en un archivo. Al cargar, lo leés y reconstruís todo. Godot trae funciones para escribir y leer archivos (`FileAccess`) y para convertir a/desde JSON (`JSON.stringify` / `JSON.parse_string`).

Mini-ejercicio

Abrí `world_map.json` y agregá un lugar nuevo (inventá nombre, tipo y una posición entre 0 y 1). Guardá, corré el juego y abrí el mapa con M: tu lugar debería aparecer. Cambiaste el contenido del juego sin escribir una línea de código.

TileMaps: construir el mundo

El mundo no se dibuja poniendo miles de imágenes a mano: se "pinta" con un **TileMap**. Un TileMap usa un **TileSet** (una grilla de tiles: pasto, camino, agua, pared) y te deja pintar el mapa como en el Paint, colocando tiles en una cuadrícula.

Por qué es tan eficiente

- Un mapa gigante pesa poquísimo (solo guarda "en la celda X va el tile número Y").
- Las colisiones se definen una vez por tile (la pared choca; el pasto no) y valen para todo el mapa.
- **Autotiling**: Godot puede elegir solo el borde correcto (esquinas, orillas) según los tiles vecinos. Magia para hacer costas y caminos.

En nuestro juego

Ahora el piso es un rectángulo verde liso (placeholder). El próximo gran paso de programación + arte juntos es reemplazarlo por un **TileMap** real: el artista dibuja el TileSet (pasto, camino, árboles, paredes de casas) y el programador arma el mapa y le pone colisiones. Ahí el juego empieza a tener cara de juego.

Mini-ejercicio

En Godot, agregá un nodo `TileMap` a una escena de prueba, creá un TileSet con 2 tiles improvisados (aunque sean dos cuadrados de distinto color) y pintá un pequeño mapa. No hace falta que quede lindo: es para entender el flujo de pintar con tiles.

Ruta Arte

Esta ruta es para quien le dé la cara al juego. Buena noticia: el pixel art es de los estilos más accesibles para empezar, porque trabajás con pocos píxeles y no necesitás saber dibujar "realista". Mala noticia: es engañoso, la simpleza tiene sus reglas. Vamos de menor a mayor dificultad.

Fundamentos de pixel art

El pixel art es dibujar colocando píxeles de a uno, en una resolución baja. La gracia y la dificultad están en lograr mucho con poco.

Los conceptos clave

- **Resolución del tile:** el tamaño base de la grilla. Para nuestro juego conviene algo chico y estándar: **16×16** o **32×32** píxeles por tile. Hay que elegir uno y respetarlo en todo el juego.
- **Paleta:** el conjunto limitado de colores. Menos colores = más coherencia y más "estilo". Para el tono de mundo colapsado nos sirven verdes/grises apagados con algún acento fuerte (rojo del peligro).
- **Legibilidad:** a tamaño chico, cada cosa tiene que *leerse* de un vistazo. El personaje se tiene que distinguir del fondo al instante; un zombie, distinguirse del personaje.
- **Contraste y silueta:** si tapás el dibujo y solo ves su silueta negra, ¿se entiende qué es? Esa es la prueba de un buen sprite.

Regla de oro para empezar

Elegí **una** resolución (ej. 32×32) y **una** paleta (16-32 colores) y no las cambies. La coherencia importa más que la perfección de cada dibujo. Un juego con arte "simple pero coherente" se ve profesional; uno con arte "hermoso pero disparejo" se ve amateur.

En nuestro juego

Hoy todo son cuadrados de colores: vos sos azul, el zombie rojo oscuro, la comida un rombo. Ese es el "placeholder" que hay que reemplazar. La primera misión de arte es definir la paleta y la resolución, y dibujar el primer tile de pasto y el primer sprite del personaje.

Mini-ejercicio

Armá un moodboard (en Figma o un tablero de imágenes) con 10 capturas de juegos pixel art que te gusten para este estilo (mirá Project Zomboid, Core Keeper, Zelda clásicos). Anotá qué resolución y paleta parecen usar. Es la base para decidir la nuestra.

Aseprite en la práctica

Aseprite es el programa estándar para pixel art (es pago pero barato; hay alternativas gratis como **LibreSprite** o **Piskel**). Está hecho específicamente para esto, así que hace fácil lo que en Photoshop sería un dolor.

Lo mínimo que hay que dominar

- **Lápiz y borrador** a nivel de píxel, y el **balde** para rellenar.
- **Paleta**: definir tus colores y trabajar solo con esos.
- **Capas (layers)**: dibujar el personaje en una capa y la sombra en otra, para editarlas por separado.
- **Frames y animación**: Aseprite tiene una línea de tiempo abajo; cada frame es un dibujo, y al reproducirlos se ve la animación.
- **Exportar**: sacar el dibujo como PNG, o como "sprite sheet" (todas las poses en una imagen) para importar en Godot.

Mini-ejercicio

Dibujá en 32×32 un sprite simple del personaje visto desde arriba (cabeza, hombros, sin detalle fino). Guardalo como PNG. No importa si es feo: el objetivo es aprender el flujo dibujar → exportar.

Tilesets y autotiling

Un **tileset** es tu "caja de piezas" para construir el mapa: pasto, camino, agua, pared, piso de casa. La clave es que las piezas **calcen entre sí** (los bordes tienen que continuar de un tile al siguiente).

Tiles que conectan

Para que un camino o una costa se vea natural, se dibujan las variantes de borde: recto, esquina, curva. Godot puede elegir las solo (autotiling) si se lo configura, pero **vos las tenés que dibujar** primero. Es la parte más técnica del arte de tiles.

En nuestro juego

Necesitamos, como mínimo para el primer mapa jugable: pasto, tierra/camino, agua (para la costa y la pesca), árboles/arbustos, y paredes/piso de casas para el looteo. Con eso ya se puede armar una porción de pueblo + bosque + costa.

Mini-ejercicio

Dibujá un tile de pasto y un tile de camino de tierra (32×32) que se vean bien uno al lado del otro. Poné 4 copias juntas en Aseprite y fijate si "se nota el corte" o si fluyen. Ajustá los bordes hasta que fluyan.

Personajes y animación

Un personaje vivo necesita al menos dos animaciones: **idle** (quieto, respirando) y **walk** (caminando). En top-down, además, suele haber una versión por dirección (arriba, abajo, izquierda, derecha).

Principios de animación en pixel art

- **Pocos frames alcanzan:** una caminata de 4-6 frames ya se ve bien.
- **El "bounce":** un leve sube-y-baja del cuerpo al caminar da vida.
- **Consistencia:** el personaje tiene que ocupar el mismo lugar en cada frame (si "salta" de posición, se ve mal).

En nuestro juego

El jugador ya tiene la variable `facing` (hacia dónde mira) en el código. Cuando el arte esté listo, el programador conecta cada animación (idle/walk por dirección) a ese estado. Arte y programación se encuentran acá: el artista entrega el sprite sheet, el programador lo enchufa con un `AnimationPlayer` o `AnimatedSprite2D`.

Mini-ejercicio

Tomá tu sprite del personaje y hacé una caminata de 4 frames (mové las piernas/brazos apenas). Reproducila en Aseprite. Si "camina", ganaste. Exportala como sprite sheet.

Interfaz (UI/HUD) con Figma

La **UI** (interfaz) es todo lo que no es el mundo: las barras de vida/hambre, el inventario, los menús. El **HUD** es la parte que se ve mientras jugás. Conviene **diseñarla en Figma antes de programarla**: mover cajas en Figma es gratis y rápido; rehacerla en código es caro.

Qué hace Figma acá

- Maquetar cómo se ven y se ordenan las pantallas (menú principal, inventario, HUD).
- Probar variantes rápido sin tocar el juego.
- Pasarle al programador una referencia clara de tamaños, posiciones y colores.

En nuestro juego

El HUD actual (barras de salud y hambre + "Comida: 0") es funcional pero feo, hecho directo en Godot. El plan es rediseñarlo en Figma (que se vea acorde al mundo de zombies) y después reconstruirlo. Ojo: Figma es para *diseñar* la interfaz; el pixel art final de los íconos igual se dibuja en Aseprite.

Mini-ejercicio

En Figma, diseñá una pantalla de inventario simple: una grilla de casillas, un título, y la barra de vida/hambre arriba. No tiene que funcionar, solo verse. Es tu primer "plano" de UI.

PARTE 3 · DISEÑO + AUDIO

Ruta Diseño + Audio

Esta ruta es para quien piensa *por qué* el juego es divertido y le pone el sonido. Es menos "técnica" en herramientas, pero es la que decide si el juego engancha o aburre. No se programa ni se dibuja: se piensa, se prueba y se ajusta.

Game design: que sea divertido y justo

El **game design** es el diseño de las reglas y la experiencia. No es "tener ideas", es **tomar decisiones** sobre cómo se siente jugar.

Conceptos que hay que manejar

- **Core loop:** el ciclo que el jugador repite (explorar → juntar → sobrevivir → mejorar la base → repetir). Si ese ciclo es satisfactorio, el juego funciona.
- **Progresión:** cómo el jugador se vuelve más capaz con el tiempo (mejores herramientas, más habilidad). Sin progresión, aburre.
- **Curva de dificultad:** el desafío tiene que subir de a poco. Ni tan fácil que aburra, ni tan difícil que frustre.
- **Tensión y alivio:** momentos de peligro (noche, zombies) seguidos de momentos de calma (armar la base de día). El contraste es lo que hace memorable un survival.
- **Economía de recursos:** qué tan escasa es la comida, la munición, los materiales. Es la perilla que define si el juego es relajado o angustiante.

En nuestro juego

Nuestro pilar es "tensión sostenida, no sustos baratos". Eso es una decisión de diseño: el miedo viene de *administrar* (¿corro y hago ruido para llegar rápido, o me agacho y voy lento pero seguro?), no de que salte un zombie de la nada. El rol de diseño ajusta constantemente esas perillas: velocidad del hambre, ruido de cada acción, daño del zombie, cuánta comida hay. Todo eso son números en el código (`hunger_decay_per_second`, `run_noise`, `attack_damage` ...) que se pueden tocar y probar.

Mini-ejercicio

Jugá el prototipo 10 minutos y anotá: ¿en qué momento te aburríste? ¿en qué momento sentiste tensión? ¿el hambre baja muy rápido o muy lento? Esas notas son *exactamente* el trabajo de un diseñador: observar y proponer ajustes.

Diseño de niveles y del mapa

Diseñar el mapa no es solo dibujarlo (eso es arte): es decidir **dónde va cada cosa y por qué**, para que explorar sea interesante.

Preguntas que se hace el diseñador de niveles

- ¿Dónde empieza el jugador y hacia dónde lo "invita" a ir el mapa?
- ¿Dónde están los recursos buenos, y qué riesgo tienen alrededor? (Recompensa alta = peligro alto.)
- ¿Hay lugares memorables que sirvan de referencia para orientarse?
- ¿El camino entre zonas es interesante o es un paseo aburrido?

En nuestro juego

Ya tenemos la estructura del mapa diseñada (ver `docs/WORLD_MAP.md`): un **hub central** (Refugio Central) del que salen caminos hacia puntos de interés, cada uno con un rol: un puesto militar (armas, más peligro), un campamento maderero (madera), zonas de cultivo (comida), un cementerio infestado (alto riesgo/alto botín). Ese "alto riesgo = alta recompensa" es diseño de niveles puro.

Mini-ejercicio

Agarrá el mapa de `world_map.json` y proponé un lugar nuevo: ¿qué ofrece?, ¿qué peligro tiene?, ¿por qué el jugador querría ir? Escribilo en 3 líneas. Así se diseña un punto de interés.

Audio: sonido y música (y el ruido como mecánica)

El sonido es la mitad de la inmersión (próba jugar cualquier juego de terror sin audio: no da nada de miedo). Hay dos tipos:

- **SFX (efectos):** pasos, golpes, agarrar un objeto, el gruñido del zombie.
- **Música:** el ambiente. Un tema calmo de día, uno tenso de noche.

De dónde sacar audio sin gastar

Hay bancos de sonidos y música **libres o royalty-free** (Freesound, OpenGameArt, música con licencia Creative Commons). **Ojo con las licencias** (ver el capítulo de seguridad/legal): algunos piden dar crédito, otros no permiten uso comercial. Anotá siempre de dónde sacaste cada sonido.

En nuestro juego, el ruido no es solo sonido: es una mecánica

Correr hace ruido y **atrae zombies** (es el círculo punteado de la captura). Entonces el audio y el diseño se cruzan: cuando el jugador corre, además de *escuchar* los pasos fuertes, el juego *usa* ese ruido para que los zombies lo detecten. El sonido de pasos debería reflejar eso (pasos silenciosos agachado, fuertes corriendo). Es un detalle que hace al juego coherente.

Mini-ejercicio

Buscá en Freesound 3 sonidos que nos servirían (pasos, un gruñido de zombie, agarrar un objeto) y fijate la licencia de cada uno. Armá una lista con el nombre, el link y la licencia. Ese es el primer ladrillo de la biblioteca de audio del juego.

Temas avanzados (un vistazo para todos)

Estos temas son para **más adelante** (fases futuras del proyecto). No hace falta dominarlos ahora, pero sí conviene que los tres sepan que existen y de qué se tratan, sobre todo el multijugador, que es lo más difícil de todo el proyecto.

Buenas prácticas y organización del código

A medida que el juego crece, la diferencia entre un proyecto que avanza y uno que se traba es el **orden del código**.

- **Nombres claros:** `attack_damage` se entiende; `x` no. El código se lee mucho más de lo que se escribe.
- **Una cosa, un lugar:** si la fórmula del daño está en 5 lugares, cambiarla es un infierno. Ponela en uno solo.
- **Componentes chicos** (como `needs_component.gd`) en vez de scripts gigantes.
- **Comentarios donde el "por qué" no es obvio** (no para explicar lo evidente).
- **Probar seguido:** correr el juego después de cada cambio, no acumular 20 cambios y después no saber cuál rompió.

💡 Rendimiento en 2D

Los juegos 2D rara vez tienen problemas de rendimiento si no hacés cosas locas (miles de nodos, cálculos pesados cada frame). No te obsesiones con optimizar antes de tiempo: primero que funcione y sea divertido, después se optimiza *solo lo que haga falta*.

Multijugador: lo más difícil

El cooperativo es el sueño del proyecto, y también **lo más complejo de programar**. Por eso en el plan lo dejamos para el final, después de que el juego solo ya sea divertido.

Por qué es tan difícil

En un juego de un jugador, hay una sola computadora que sabe todo. En multijugador hay **varias computadoras que tienen que estar de acuerdo** sobre qué está pasando, comunicándose por internet (con demoras, cortes, etc.). Los problemas típicos:

- **Sincronización:** que todos vean al zombie en el mismo lugar al mismo tiempo.
- **Autoridad:** ¿quién decide "la verdad"? Normalmente un **servidor** manda, y los jugadores (**clientes**) le hacen caso. Esto se llama modelo **cliente-servidor**.
- **Latencia:** la demora de internet hace que las cosas lleguen tarde; hay que disimularla.

⚠ La regla de seguridad más importante del multijugador

Nunca confíes en el cliente. La computadora de cada jugador puede ser modificada para hacer trampa. La lógica importante (cuánta vida tenés, cuánto daño hacés, qué recursos tenés) la tiene que validar el servidor, no el cliente. Si confiás en lo que manda cada jugador, cualquiera puede decir "tengo 9999 de vida" y el juego le cree.

🎮 En nuestro juego

Godot 4 trae un sistema de **multijugador de alto nivel** que ayuda bastante (nodos que se sincronizan casi solos). Aun así, es un mundo aparte. Por eso el código lo estamos escribiendo desde ahora *pensado* para que mañana sea más fácil separarlo en cliente/servidor, aunque todavía no lo hagamos.

Seguridad aplicada

Mientras el juego sea **de un jugador y sin conexión**, la seguridad casi no es un tema. Todo cambia cuando hay internet de por medio. Resumen de lo que hay que cuidar (el detalle está en el plan del proyecto):

- **No confiar en el cliente** (ya lo vimos): vale para trampas y para exploits.
- **Validar todo lo que llega por la red**: datos mal formados no deben crashear ni, peor, permitir ejecutar código.
- **Datos de los jugadores**: si algún día hay cuentas, nunca guardar contraseñas en texto plano; mejor usar login de terceros (Steam) y no manejarlas nosotros.
- **Conexiones directas (P2P)**: los jugadores se ven las IP entre sí, que es un dato personal y un vector de ataque. Conviene avisarlo o usar un servidor intermedio.
- **Chat / contenido**: si hay chat, tiene que haber forma de moderar y reportar.

💡 Tranquilos por ahora

Todo esto es de la Fase 3-4. Hoy, con un juego offline que no recolecta ningún dato, nuestra exposición es prácticamente cero. Está acá para que sepan que existe y no los agarre desprevenidos cuando lleguemos al online.

Exportar y publicar en itch.io

Cuando haya algo mostrable, Godot puede **exportar** el juego a un ejecutable (un `.exe` para Windows, versiones para Mac/Linux). Eso es un archivo que cualquiera puede correr sin tener Godot.

El camino a itch.io

1. En Godot, agregar las "plantillas de exportación" y configurar el export para Windows (y Linux/Mac si se quiere).
2. Exportar → queda un ejecutable (o un ZIP con el juego).
3. Crear una cuenta en **itch.io** (gratis), crear la página del juego, subir el ZIP, poner descripción, capturas y el rating de contenido.
4. ¡Publicado! Se puede poner gratis o "pagá lo que quieras".

En nuestro juego

Como somos sin fines de lucro, itch.io es nuestro destino natural: publicar gratis, sin costos. Steam queda como opción lejana y opcional (tiene un costo de USD 100). Una buena práctica: subir versiones tempranas para amigos/testers mucho antes del "lanzamiento".

Mini-ejercicio (para el que tome programación/publicación)

Cuando tengas el proyecto abierto en Godot, probá exportar el prototipo actual a un ejecutable de tu sistema operativo y correlo fuera de Godot. Ver el juego andando como un programa "de verdad" es muy motivador.

APÉNDICES

Material de apoyo

Apéndice A

Cronograma de estudio de 1 semana

Una propuesta de cómo repartir la semana. Los días 1-3 son para todos (núcleo común). Del día 4 en adelante, cada uno sigue su ruta. Ajustenlo a su ritmo: es una guía, no una obligación.

Día	Todos	Programación	Arte	Diseño+Audio
1	Caps 1 y 2 (qué es un juego + nuestro juego). Instalar Godot y correr el prototipo (F5). Jugarlo un rato.			
2	Cap 3 (Git/GitHub): crear cuentas, acceso al repo, practicar pull/commit/push. Cap 5 (armar el Notion).			
3	Cap 4 (Godot y nuestro proyecto): abrir <code>Main.tscn</code> , explorar el árbol de nodos, tocar alguna variable y ver el efecto.			
4	—	Caps 6-7 (GDScript, nodos y señales) + ejercicios	Caps 14-15 (fundamentos pixel art + Aseprite) + ejercicios	Cap 19 (game design) + jugar y anotar
5	—	Caps 8-9 (input/movimiento, colisiones) leyendo el código real	Cap 16 (tilesets) + primer tile de pasto/camino	Cap 20 (diseño de niveles) + proponer un POI
6	—	Caps 10-11 (componentes, IA del zombie)	Cap 17 (personaje + caminata)	Cap 21 (audio) + juntar sonidos libres
7	Cada uno cierra su ruta (Prog: caps 12-13; Arte: cap 18 UI; D+A: repaso). Vistazo grupal a la Parte 4 (avanzado). Juntarse, repartir las primeras tareas reales en Notion y arrancar.			

💡 El día 7 es el más importante

No es para estudiar más: es para **juntarse y planificar**. Con todo fresco, definan las primeras 2-3 semanas de trabajo real: qué hace cada uno, cuál es el primer objetivo concreto (sugerencia: reemplazar el placeholder por un mini-mapa con tiles reales y el personaje animado). Ahí empieza el juego de verdad.

Glosario de términos

Término	Significado
Assets	Los recursos del juego: imágenes, sonidos, música, fuentes.
Build / Export	Generar el juego como programa ejecutable para distribuir.
Commit / Push / Pull	Guardar cambios (commit), subirlos (push), bajarlos (pull) en Git.
Delta	Tiempo transcurrido desde el frame anterior, en segundos.
FPS	Frames por segundo; cuántas veces por segundo se dibuja el juego.
Frame	Un "cuadro": una vuelta del game loop.
Game loop	El bucle que corre el juego: leer input, actualizar, dibujar.
GScript	El lenguaje de programación de Godot.
HUD	La interfaz que se ve mientras jugás (barras, contadores).
Hitbox / Colisión	La forma que usa un objeto para detectar contactos.
JSON	Formato de texto para guardar datos ordenados.
Máquina de estados	Sistema donde algo está en un estado a la vez y cambia según reglas.
Nodo	La pieza básica de Godot; hace una cosa.
Placeholder	Arte/contenido provisorio hasta tener el definitivo.
Señal (signal)	Aviso que un nodo emite y otros escuchan.
Sprite	Imagen 2D que se dibuja en pantalla.
Tile / TileMap / TileSet	Azulejo / el mapa hecho de azulejos / la caja de azulejos disponibles.
Cliente-servidor	Modelo de multijugador donde el servidor manda y los clientes obedecen.

Recursos gratis para seguir aprendiendo

Godot y programación

- **Documentación oficial de Godot** (docs.godotengine.org): tiene un tutorial "tu primer juego 2D" buenísimo para empezar.
- **GDQuest** y **Brackeys** (YouTube): tutoriales de Godot muy claros y gratuitos.
- El propio **código de nuestro juego** ([game/scripts/](#)): el mejor material de estudio, porque es el que vamos a usar.

Pixel art

- **Pixel art de cero** — tutoriales de *Pixel Pete*, *AdamCYounis* (YouTube).
- **Lospec** (lospec.com): paletas gratis y guías de pixel art.
- **LibreSprite / Piskel**: alternativas gratis a Aseprite.

Assets libres (arte y sonido)

- **OpenGameArt.org** — arte y audio libre (revisá la licencia de cada uno).
- **Kenney.nl** — packs de assets gratis y de excelente calidad.
- **Freesound.org** — efectos de sonido (revisá licencias).
- **itch.io** — muchísimos asset packs gratis o baratos, y donde vamos a publicar.

Siempre revisá la licencia

Antes de usar cualquier asset descargado, fijate qué permite su licencia (uso comercial, si hay que dar crédito, si se puede modificar) y anotalo en un archivo `CREDITS.md`. Es la regla que nos mantiene fuera de problemas legales.

Última cosa

No se estresen por entender todo a la perfección en una semana. Con este libro tienen el mapa completo del terreno. El conocimiento de verdad va a llegar **haciendo el juego**, equivocándose y arreglando. Lo importante es empezar. ¡Éxitos, equipo! 